研究文档

多模态神经算子代码导航

学术与技术文档

BURGERS EQUATION · OPERATOR LEARNING · REPRODUCIBILITY

日期

2026 年 11 月 9 日

项目

多模态神经算子研究

作者

项目组

目录

核心代码位置

架构组成

信息流

关键函数

关键参数调节

代码统计

快速开始修改

文件关系

核心代码位置

主文件: `code/demo_burgers_dataset.py` (3950行)

架构组成

1. 配置类 (Line 36-130)

```
@dataclass
class DatasetConfig:
    # 关键参数
    use_cross_attention: bool = True        # 交叉注意力开关
    manifold_arch: str = "mlp"              # manifold编码器架构
    manifold_hidden: int = 128              # manifold隐层维度
    dropout: float = 0.05                   # Dropout正则化
    physics_weight: float = 0.05            # 物理约束权重
    sensor_count: int = 0                   # 传感器数量
    ...
```

2. 交叉注意力机制 (Line 452-490)

```
class ManualAttention(nn.Module):
    """交叉注意力: 传感器信息和PDE约束的动态融合"""
    def forward(self, tokens, context=None):
        # 动态调整物理权重
        # 融合多模态信息
```

位置: 第452行

3. Manifold编码器 (Line 531-625)

```
class ManifoldAutoencoder(nn.Module):
    """学习隐式PDE求解算子的低维表示"""

class ConvManifoldAutoencoder(nn.Module):
    """卷积版本的Manifold编码器(更强大)"""
```

位置: 第531行-625行

4. Transformer编码器 (Line 510-530)

```
class TransformerEncoder(nn.Module):
    """多头自注意力编码器"""
    def forward(self, tokens):
        # 处理sensor tokens序列
        # 提取高阶特征
```

位置: 第510行

5. 神经算子主体 (Line 700+)

```
class NeuralOperatorModel(nn.Module):
    """完整的混合PINN架构"""

    def __init__(self, cfg):
        # Manifold编码器
        self.manifold = build_manifold(cfg)

        # 交叉注意力
        self.cross_attention = ManualAttention( ... )

        # Transformer编码器
        self.transformer = TransformerEncoder( ... )

        # 查询解码器
        self.query_decoder =  ...
```

信息流

```
输入: u(x,t) + 传感器观测 + PDE约束
  ↓
[1] Manifold编码 (line 531+)
    └─ 降维: (batch, nx, nt) → (batch, latent_dim)
  ↓
[2] 交叉注意力融合 (line 452+)
    ├─ 融合多模态传感器信息
    └─ 动态调整physics_weight
  ↓
[3] Transformer特征提取 (line 510+)
    └─ 多头自注意力处理token序列
  ↓
[4] 查询解码 (line 664+)
    └─ 生成u(x,t,c)预测
  ↓
[5] PDE损失计算 (line 2000+)
    ├─ 数据项: MSE(u_pred, u_true)
    └─ 物理项: ∂u/∂t + u∂u/∂x - ν∂²u/∂x²
  ↓
输出: u_pred(x,t) + PDE residual
```

关键函数

函数名	行号	功能
`build_manifold()`	627	构建manifold编码器
`create_model()`	~3000	创建完整模型
`train()`	~2500	训练循环
`evaluate_split()`	~2000	评估指标
`compute_pde_loss()`	~1800	PDE残差计算

关键参数调节

交叉注意力

```
# 启用/禁用
--use-cross-attention      # 启用交叉注意力
--no-cross-attention       # 禁用(降级为简单融合)
```

Manifold编码器

```
# 架构选择
--manifold-arch mlp        # MLP版本(轻量)
--manifold-arch conv       # 卷积版本(强大)

# 容量控制
--manifold-hidden 128      # 隐层维度
--latent-dim 16           # 潜在空间维度
```

物理权重

```
# 调整物理约束强度
--physics-weight 0.1       # 训练时权重
--validation-physics-weight 0.1  # 验证时权重

# 动态调整由交叉注意力处理
```

代码统计

```
总行数: 3950
关键类: 15+
主要模块:
  - Manifold编码 (100行)
  - 交叉注意力 (50行)
  - Transformer (100行)
  - 神经算子主体 (200行)
  - 训练循环 (300行)
  - PDE损失计算 (200行)
```

快速开始修改

要修改架构

1. 打开 `demo_burgers_dataset.py`
2. 找到 `class NeuralOperatorModel` (~700行)
3. 修改 `__init__` 和 `forward` 方法

要修改物理约束

1. 找到 `compute_pde_loss()` (~1800行)
2. 修改PDE方程项
3. 调整 `physics_weight`

要修改数据增强

1. 打开 `augment_burgers.py`
2. 修改增强策略
3. 重新生成数据

文件关系

```
demo_burgers_dataset.py (主训练脚本)
├─ 定义: NeuralOperatorModel
├─ 定义: DatasetConfig
├─ 包含: 训练循环
└─ 调用: augment_burgers.py (数据加载)

augment_burgers.py (数据增强)
├─ 空间增强
├─ 时间增强
└─ Diffusion验证

train_baseline_resnet.py (数据驱动基线)
└─ 用于对比
```

当前最优方案: v_aug3x_xl_d128_l4 - 位置: `experiments/v_aug3x_xl_d128_l4/` - 性能: PDE RMS = 0.001059 - 代码配置: `--preset wide --d-model 128 --n-layers 4`